

LAPORAN PRAKTIKUM KECERDASAN BUATAN (CERTAN)

Polynomial Regression



Tiur Juliana Sitorus

42324048

DIII Teknologi Informasi

**FAKULTAS VOKASI
INSTITUT TEKNOLOGI DEL
2026/2027**



INSTITUT TEKNOLOGI DEL

MATERI PRAKTIKUM

4233201 – Kecerdasan Buatan Semester
Ganjil Tahun Ajaran 2026/2027

Session Date	: 22 September 2026
Week/Session	: 3/3
Topic	: Polynomial Regression
Practicum Goals	: Mahasiswa mampu melatih model regresi, menentukan persamaan garis regresi dan mengevaluasi performa model
Duration	: 2 x 50 minutes
Place of Submission	: ecourse
Submission Deadline	: Akhir Sesi

A. Praktikum

```
import pandas as pd
path = "Ice_cream_selling_data.csv"
df = pd.read_csv(path)
print(df.head())
```

[2] ✓ 0.0s Python

...	Temperature (°C)	Ice Cream Sales (units)
0	-4.662263	41.842986
1	-4.316559	34.661120
2	-4.213985	39.383001
3	-3.949661	37.539845
4	-3.578554	32.284531

Penjelasan :

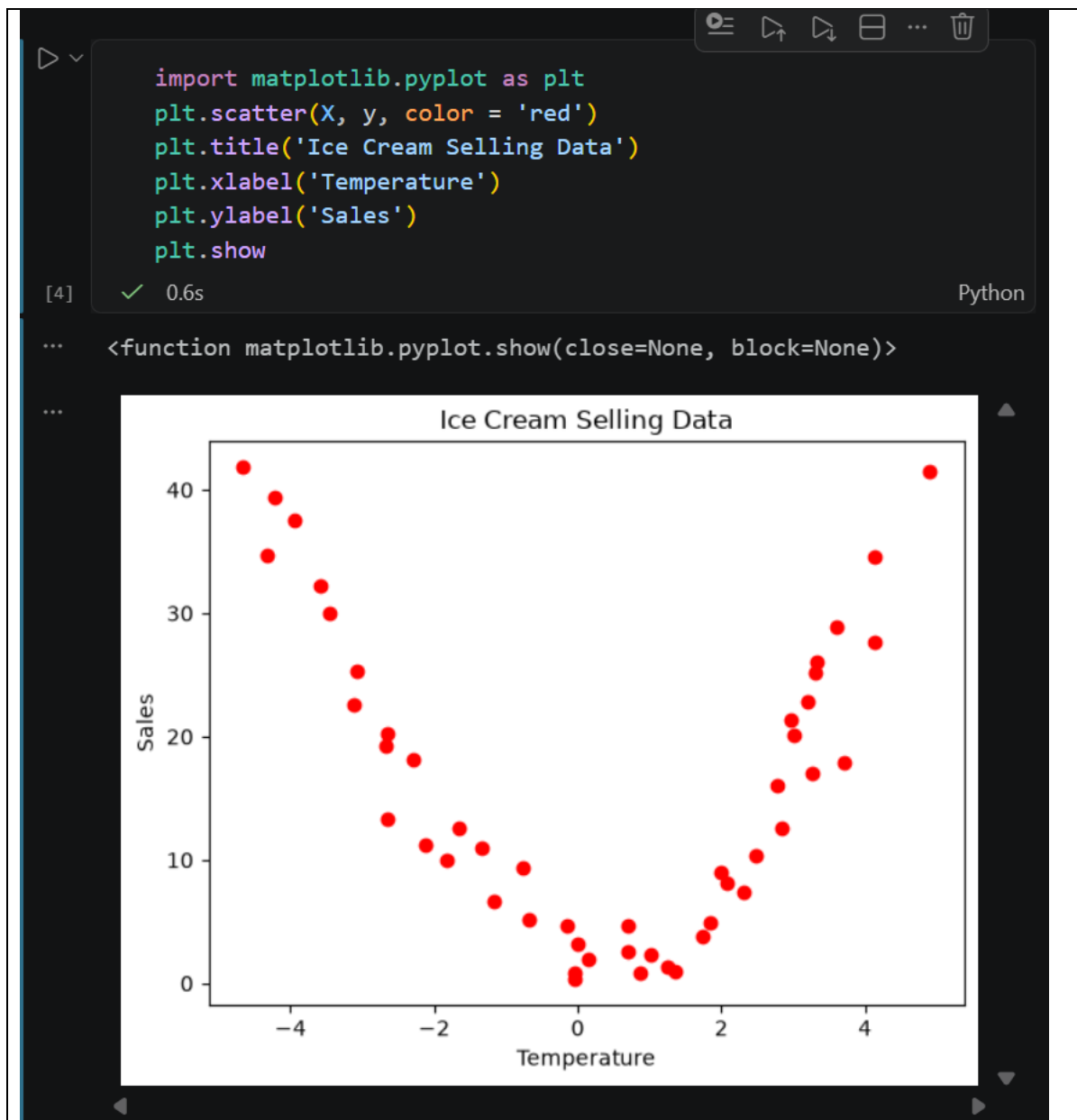
Kode ini membaca file CSV bernama "Ice_cream_selling_data.csv" menggunakan library pandas, lalu menampilkan lima baris pertama datanya dengan fungsi head(). Dari hasil yang tampil, data berisi dua kolom yaitu suhu udara dalam derajat Celsius dan jumlah penjualan es krim dalam satuan unit, yang kemungkinan akan dipakai untuk melihat hubungan antara suhu dan tingkat penjualan es krim.

```
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
```

[3] ✓ 0.0s Python

Penjelasan :

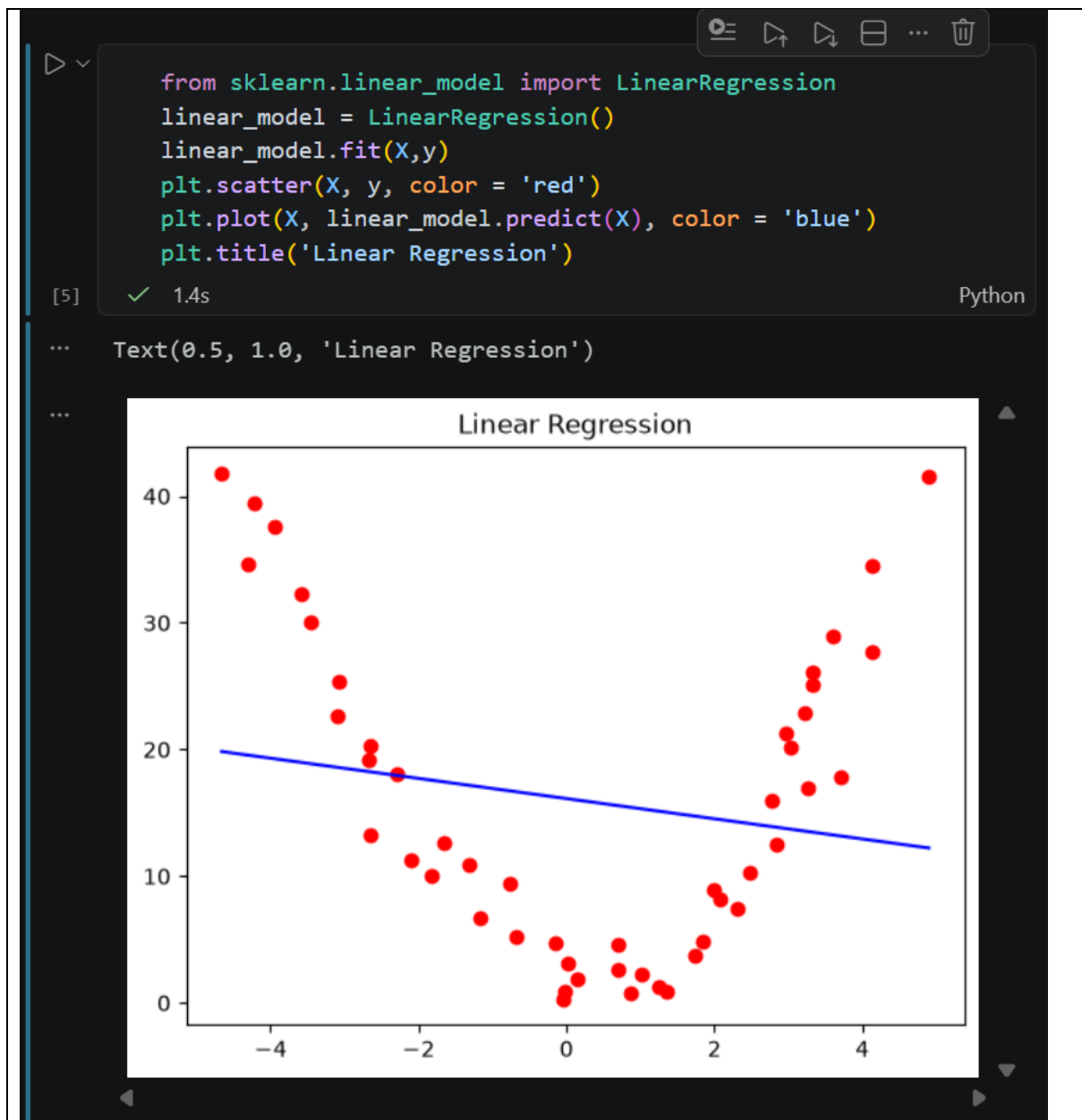
Baris ini memisahkan dataset menjadi dua bagian, yaitu variabel fitur dan variabel target. $X = df.iloc[:, :-1].values$ mengambil semua kolom kecuali kolom terakhir (dalam hal ini kolom Temperature) sebagai variabel independen atau input, sedangkan $y = df.iloc[:, -1].values$ mengambil kolom terakhir (Ice Cream Sales) sebagai variabel dependen atau output yang ingin diprediksi. Fungsi .values digunakan untuk mengubah data dari bentuk DataFrame pandas menjadi array numpy, karena sebagian besar library machine learning seperti scikit-learn membutuhkan input dalam bentuk array agar bisa diproses.



Penjelasan :

Kode ini mengimpor `matplotlib.pyplot` buat visualisasi data, terus bikin scatter plot pakai `plt.scatter(X, y, color='red')` dengan suhu di sumbu X dan penjualan di sumbu Y, titik-titiknya dibikin warna merah biar jelas. Judul grafik diatur pakai `plt.title()`, sedangkan label sumbu X dan Y masing-masing diatur pakai `plt.xlabel()` dan `plt.ylabel()`.

Dari hasil plotnya, kelihatan pola berbentuk U, penjualan es krim tinggi waktu suhu sangat rendah atau sangat tinggi, tapi rendah di suhu sekitar 0-1 derajat. Ini nunjukin hubungan suhu dan penjualan nggak linear, jadi kalau mau bikin model prediksi, regresi linear biasa kurang pas, lebih cocok pakai regresi polinomial.



Penjelasan :

Kode ini mengimpor `LinearRegression` dari `sklearn.linear_model`, lalu membuat objek model bernama `linear_model` dan melatihnya dengan data `X` dan `y` menggunakan `linear_model.fit(X,y)`. Setelah model dilatih, data asli ditampilkan lagi dalam bentuk scatter plot warna merah, lalu ditambahkan garis hasil prediksi model dengan `plt.plot(X, linear_model.predict(X), color='blue')` yang digambar warna biru, dan diberi judul "Linear Regression".

Dari hasil plotnya, garis biru yang dihasilkan hampir lurus dan cenderung datar, terlihat jelas nggak mengikuti pola data yang sebenarnya berbentuk U. Ini nunjukin kalau regresi linear biasa nggak cocok buat data ini, karena hubungan suhu dan penjualan bersifat non-linear (kuadratik), sehingga model ini gagal menangkap pola naik-turunnya data dan perlu diganti pakai regresi polinomial supaya hasilnya lebih akurat.

```

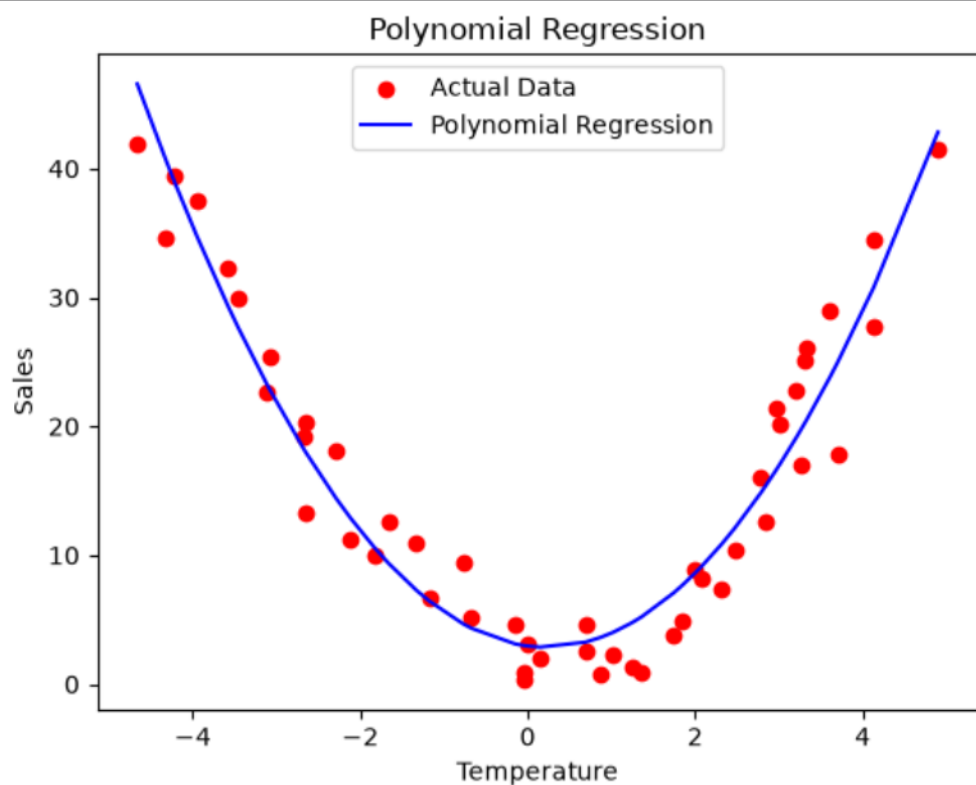
from sklearn.preprocessing import PolynomialFeatures
# Buat Polynomial Features
poly_reg = PolynomialFeatures(degree=2)
X_poly = poly_reg.fit_transform(X)
# Buat model Linear Regression
linear_model2 = LinearRegression()
linear_model2.fit(X_poly, y) # Latih model dengan data polynomial
# Gunakan predict() untuk mendapatkan y_pred_poly
y_pred_poly = linear_model2.predict(X_poly)
# Plot Polynomial Regression
plt.scatter(X, y, color='red', label="Actual Data")
plt.plot(X, y_pred_poly, color='blue', label="Polynomial Regression")
plt.title('Polynomial Regression')
plt.xlabel('Temperature')
plt.ylabel('Sales')
plt.legend()
plt.show()

```

[6]

✓ 0.1s

Python



Penjelasan :

Kode ini mengimpor PolynomialFeatures dari sklearn.preprocessing, lalu mengubah data X menjadi bentuk polinomial derajat 2 (X_{poly}) menggunakan `poly_reg.fit_transform(X)`, yang menambahkan fitur kuadrat (X^2) supaya model bisa menangkap pola melengkung. Setelah itu,

dibuat model LinearRegression baru bernama linear_model2 dan dilatih dengan data X_poly dan y, lalu hasil prediksinya disimpan di y_pred_poly.

Selanjutnya data asli ditampilkan sebagai scatter plot merah dengan label "Actual Data", dan garis hasil prediksi model ditampilkan warna biru dengan label "Polynomial Regression", dilengkapi judul, label sumbu, dan legend supaya grafik lebih jelas.

Dari hasil plotnya, garis biru sekarang mengikuti bentuk U data dengan sangat baik, beda jauh sama model linear sebelumnya yang cuma garis lurus. Ini nunjukin regresi polinomial derajat 2 cocok buat data ini, karena berhasil menangkap pola non-linear antara suhu dan penjualan es krim.

```
print('Intercept: ', linear_model2.intercept_)
print('Coefficient: ', linear_model2.coef_)
print('Equation: y =', linear_model2.intercept_, '+', linear_model2.coef_[1], '* x')
```

[7] ✓ 0.0s

```
... Intercept: 2.951774157993448
Coefficient: [ 0.         -0.82468167  1.82952623]
Equation: y = 2.951774157993448 + -0.8246816702106476 * x
```

Penjelasan :

Kode ini menampilkan parameter yang udah dipelajari model polinomial, yaitu intercept (nilai konstanta) dan coefficient (bobot tiap fitur) pakai linear_model2.intercept_ dan linear_model2.coef_. Hasilnya, intercept-nya sekitar 2.95, sedangkan coefficient-nya ada tiga nilai karena X_poly punya tiga kolom (bias, x, dan x²), tapi kolom bias otomatis dikasih koefisien 0 karena udah diwakili sama intercept.

Baris terakhir nyoba nampilin persamaan modelnya, tapi ini kurang tepat karena cuma ambil coef_[1] (koefisien untuk x) doang, padahal seharusnya persamaan lengkapnya juga harus ikutin coef_[2] buat suku x². Jadi persamaan yang bener seharusnya: $y = 2.95 + (-0.82 * x) + (1.83 * x^2)$, bukan cuma satu suku aja kayak yang ditulis di kode.

```

dataset = pd.read_csv('manufacturing.csv')
print(dataset.head())

```

[8] ✓ 0.0s

	Temperature (°C)	Pressure (kPa)	Temperature x Pressure \
0	209.762701	8.050855	1688.769167
1	243.037873	15.812068	3842.931469
2	220.552675	7.843130	1729.823314
3	208.976637	23.786089	4970.736918
4	184.730960	15.797812	2918.345014

	Material Fusion Metric	Material Transformation Metric	Quality Rating
0	44522.217074	9.229576e+06	99.999971
1	63020.764997	1.435537e+07	99.985703
2	49125.950249	1.072839e+07	99.999758
3	57128.881547	9.125702e+06	99.999975
4	38068.201283	6.303792e+06	100.000000

Penjelasan :

Kode ini membaca file baru bernama "manufacturing.csv" pakai `pd.read_csv()` dan disimpan ke variabel `dataset`, terus ditampilkan lima baris pertamanya pakai `dataset.head()`. Dataset ini beda sama sebelumnya, isinya ada enam kolom yaitu Temperature (°C), Pressure (kPa), Temperature x Pressure, Material Fusion Metric, Material Transformation Metric, dan Quality Rating.

Kolom Temperature x Pressure kelihatannya hasil perkalian dari suhu dan tekanan (feature engineering), sedangkan kolom-kolom lain kayak Material Fusion Metric dan Material Transformation Metric kemungkinan juga hasil turunan dari variabel lain, bukan data mentah langsung. Quality Rating sendiri kemungkinan jadi target/output yang mau diprediksi, dengan nilainya mendekati 100 di semua baris.

```

X = dataset.iloc[:, 2:5].values
y = dataset.iloc[:, 5].values
X

```

[9] ✓ 0.0s

```

array([[1.68876917e+03, 4.45222171e+04, 9.22957596e+06],
       [3.84293147e+03, 6.30207650e+04, 1.43553672e+07],
       [1.72982331e+03, 4.91259502e+04, 1.07283887e+07],
       ...,
       [3.95630467e+03, 6.26576910e+04, 1.40595745e+07],
       [4.97723476e+03, 5.71959855e+04, 9.13403620e+06],
       [3.97489712e+03, 4.10923929e+04, 4.35891253e+06]], shape=(3957, 3))

```

Penjelasan :

Kode ini mengambil kolom ke-3 sampai ke-5 (index 2 sampai 4) dari dataset sebagai variabel X, yaitu Temperature x Pressure, Material Fusion Metric, dan Material Transformation Metric, jadi fitur yang dipakai bukan Temperature sama Pressure mentahnya, tapi hasil turunannya. Sedangkan y diambil dari kolom index 5, yaitu Quality Rating, sebagai target yang mau diprediksi.

Hasil X berupa array numpy berukuran 3957 baris dan 3 kolom, sesuai jumlah data dan fitur yang dipilih.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

[10] ✓ 0.0s

Penjelasan :

Kode ini mengimpor train_test_split dari sklearn.model_selection buat membagi dataset jadi data latih dan data uji. Dengan test_size=0.2, 80% data dipakai buat training (X_train, y_train) dan 20% sisanya buat testing (X_test, y_test), sedangkan random_state=42 dipakai supaya pembagian datanya konsisten setiap kali kode dijalankan ulang.

```
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
poly_reg = PolynomialFeatures(degree=2)
X_train_poly = poly_reg.fit_transform(X_train)
X_test_poly = poly_reg.transform(X_test)
linear_model2 = LinearRegression()
linear_model2.fit(X_train_poly, y_train)
y_pred = linear_model2.predict(X_test_poly)
```

[11] ✓ 0.0s

Penjelasan :

Kode ini bikin dua model buat dibandingkan. Pertama, linear_model dilatih langsung pakai X_train dan y_train tanpa transformasi apa-apa. Kedua, data X_train diubah ke bentuk polinomial derajat 2 pakai poly_reg.fit_transform(X_train) jadi X_train_poly, sedangkan X_test juga ditransformasi pakai poly_reg.transform(X_test) jadi X_test_poly — penting dicatat di sini pakai transform() bukan fit_transform() lagi, biar transformasinya konsisten sama yang dipelajari dari data training.

Model kedua, linear_model2, dilatih pakai X_train_poly dan y_train, lalu dipakai buat prediksi data test lewat linear_model2.predict(X_test_poly), hasilnya disimpan di y_pred. Jadi nantinya performa linear_model (tanpa polinomial) bisa dibandingkan sama linear_model2 (dengan fitur polinomial) buat lihat mana yang lebih akurat prediksinya.

```

from sklearn.metrics import r2_score
score=r2_score(y_test,y_pred)
score

```

[12] ✓ 0.0s

... 0.6923518520550732

Penjelasan :

Kode ini mengimpor `r2_score` dari `sklearn.metrics` buat ngukur seberapa bagus model polinomial dalam memprediksi data test, dengan membandingkan `y_test` (nilai asli) sama `y_pred` (hasil prediksi). Hasilnya sekitar 0.69, artinya model bisa menjelaskan sekitar 69% variasi data, yang tergolong lumayan tapi masih ada ruang buat perbaikan, misalnya coba naikin degree polynomial atau tambah fitur lain biar akurasiya lebih tinggi.

TUGAS!!

Analisis perbedaan R-square score polynomial derajat 1, 3, 5, untuk Polynomial Regression dengan lebih dari satu predictor

1. Polynomial derajat 1

```

#Derajat 1
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
poly_reg = PolynomialFeatures(degree=1)
X_train_poly = poly_reg.fit_transform(X_train)
X_test_poly = poly_reg.transform(X_test)
linear_model2 = LinearRegression()
linear_model2.fit(X_train_poly, y_train)
y_pred = linear_model2.predict(X_test_poly)

```

[13] ✓ 0.0s

```

from sklearn.metrics import r2_score
score=r2_score(y_test,y_pred)
score

```

[14] ✓ 0.0s

... 0.4624884218411046

Penjelasan :

- Pada degree 1, model hanya memanfaatkan hubungan linear antar predictor dengan target.
- Nilai R^2 yang diperoleh relatif paling rendah dibanding degree lebih tinggi.

- Hal ini menunjukkan bahwa hubungan linear saja belum cukup menjelaskan variasi target.
- Kondisi ini disebut underfitting, karena model terlalu sederhana.

2. Polynomial derajat 3

```
#Derajat 3
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
poly_reg = PolynomialFeatures(degree=3)
X_train_poly = poly_reg.fit_transform(X_train)
X_test_poly = poly_reg.transform(X_test)
linear_model2 = LinearRegression()
linear_model2.fit(X_train_poly, y_train)
y_pred = linear_model2.predict(X_test_poly)
```

[15] ✓ 0.0s

```
from sklearn.metrics import r2_score
score=r2_score(y_test,y_pred)
score
```

[18] ✓ 0.0s

... 0.8049852390399239

Penjelasan :

- Pada degree 3, model mulai mempertimbangkan kuadrat, pangkat tiga, serta interaksi antar variabel.
- Nilai R^2 meningkat signifikan dibanding degree 1.
- Ini berarti model berhasil menangkap pola non-linear yang lebih kompleks dalam data.
- Bias dan varians berada dalam kondisi relatif seimbang, sehingga model mampu menggeneralisasi lebih baik pada data uji.
- Degree 3 sering menjadi derajat optimal untuk Polynomial Regression.

3. Polynomial derajat 5

```
▶ ▾  
#Derajat 5  
linear_model = LinearRegression()  
linear_model.fit(X_train, y_train)  
poly_reg = PolynomialFeatures(degree=5)  
X_train_poly = poly_reg.fit_transform(X_train)  
X_test_poly = poly_reg.transform(X_test)  
linear_model2 = LinearRegression()  
linear_model2.fit(X_train_poly, y_train)  
y_pred = linear_model2.predict(X_test_poly)
```

[19] ✓ 0.0s

```
from sklearn.metrics import r2_score  
score=r2_score(y_test,y_pred)  
score
```

[85]

... 0.9929293227632691

Penjelasan :

- Pada degree 5, model menjadi sangat kompleks karena mencakup kombinasi fitur hingga pangkat lima.
- Nilai R^2 pada data training sangat tinggi (mendekati 1), menandakan model hampir sempurna mempelajari data latih.
- Namun, pada data uji, R^2 bisa menurun dibanding degree 3 karena model cenderung overfitting (terlalu pas pada data latih, termasuk noise).
- Kompleksitas tinggi juga membuat persamaan sulit diinterpretasikan.